

Def в Python (№1)

“Принятие функций”

советуется изучать в связке с проектом на гитхабе
(<https://github.com/the-wlfrm/GuideToDef>)

Функция (def) - это фрагмент кода выполняющий определенные операции и отдает результат обратно

В течении данного курса мы узнаем:

- Базовый синтаксис Def()
- Как функции могут взаимодействовать между собой

А также создадим свою мини-игру в терминале + поработаем с нашими первыми библиотеками!

Базовый синтаксис Def()

$a + b$

Базовый вид функций таков:

```
def имя_функции(аргументы):  
    тело_функции (код)
```

```
    # Возвращаем результат  
    return результат
```

```
    # Запуск функции  
    имя_функции(параметры)
```

Пример:

```
def main(a, b):  
    return a + b  
  
main(1, 2)
```

Инициализируется/создается функция main() с аргументами (a, b)

Функция возвращает сумму a + b

Вызываем функцию через название функции (main)и передаем параметры a и b

Мы передали (1, 2), а дальше функция сама производит действия (1 + 2) и возвращает ответ сразу

Чтобы мы могли увидеть результат, есть два варианта вызова функции

Варианты:

```
print(main(1, 2))  
  
result = main(1, 2)  
print(result)
```

1 - Принтим/выводим и запускаем функцию

2 - Записываем ответ функции в переменную result и далее выводим эту переменную

ДЗ - Напиши функцию, что принимает две буквы и соединяет их не путай int и str

Def в Python (№2)

“Персонаж”

*советуется изучать в связке с проектом на гитхабе
(<https://github.com/the-wlfrm/GuideToDef>)*

Как мы говорили ранее, функции выполняют какой-то фрагмент кода и возвращают результат

Сейчас мы напишем функцию, что будет отправлять “Привет!” нашему персонажу!

Для начала вспомним, как выглядят сами функции? (синтаксис)

Синтаксис функций:

```
def имя_функции(аргументы):  
    тело_функции (код)
```

```
    # Возвращаем результат  
    return результат
```

```
    # Запуск функции  
    имя_функции(параметры)
```

Так как-же создать функцию, что будет отправлять “привет!” нашему персонажу?

*Инициализируем/создаем функцию def() под названием "character" (персонаж)
Передаем ему аргумент "name" (имя)*

```
def character(name):  
  
    # Возвращаем приветствие состоящее из строк  
    # и наших аргументом (в данном случае один аргумент - name)  
    return "Привет, " + name + "!"  
  
# Вызываем функцию и сразу выводим то,  
# что возвращает функцию (не забудьте указать значение нашему аргументу!)  
print(character("Max"))
```

Вывод программы будет такой:

```
w1@MacBook-Air-W1 GuideToDef  
Привет, Max!
```

Закрепим для себя, что функции действительно полезная часть кода, что упрощает жизнь многим программистам и просто является делителем кода, за счет чего становится легче воспринимать код

Мы буквально за секунду перенесли какие-то данные в эту функцию и вернули результат действий самого фрагмента кода в функции с использование этих данных!

ДЗ - написать функцию, что выводит имя и возраст вашего персонажа
Формат: "Привет, (name)! Я знаю, что тебе (age) лет!"



Def в Python (№3)

“Юзер - наш указатель”

*советуется изучать в связке с проектом на гитхабе
(<https://github.com/the-wlfrm/GuideToDef>)*

*“ - Юзер имеет право на интерактивность
и взаимодействие с программой в целом! (Адаптивность) “*

Сейчас мы напишем код, что будет с помощью функции принимать какие-то данные и вернем результат

*Инициализируем/создаем функцию def() и называем - "yourCharacter"
Прошу заметить, мы не передаем аргументы!*

```
def yourCharacter():  
  
    # Запрашиваем у пользователя имя и возраст  
    name = input("Введите имя персонажа: ")  
    age = input("Введите возраст персонажа: ")  
    # Все, что написал пользователь  
    # поочередно передается в эти переменные!  
  
    # Выводим СРАЗУ всю информацию  
    print("Имя персонажа: " + name + ", Возраст персонажа: " + age)  
  
# Вызываем эту функцию и не передаем параметры,  
# потому что функцию сама не принимает их  
yourCharacter()
```

Да, мы можем запускать и создавать функции без аргументов.

Такие функции выполняют фрагмент кода и:

- либо возвращают результат действий (return)
- либо прямо в функции выводят какую-то информацию (print())

Результат нашего кода:

```
w1@MacBook-Air-W1 GuideToDef % /usr/local/bin/  
Введите имя персонажа: Мах  
Введите возраст персонажа: 18  
Имя персонажа: Мах, Возраст персонажа: 18
```

Def в Python (№4)

“Взаимодействие между функциями”

*советуется изучать в связке с проектом на гитхабе
(<https://github.com/the-wlfrm/GuideToDef>)*

Мы много раз цепляли тему того, что функция выполняет какие-то свои задачи в результате чего, мы можем возвращаемые данные из одной функции -> использовать в другой функции!

Наша цель в этой части:

- Создать функцию, что принимает Имя от пользователя
- Создать функцию, что отправляет Привет с использованием этого имени

Инициализируем/создаем нашу первую функцию def() и называем ее "characterName"

Опять-же, он у нас без аргументов/параметров

```
def characterName():  
  
    # Запрашиваем имя у юзера  
    name = input("Введите имя персонажа: ")  
  
    # Возвращаем переменную имени, что ввел пользователь  
    return name
```

И инициализируем/создаем нашу вторую функцию def() и называем ее "characterHello"

Он уже принимает в себя аргумент (name) - Имя

```
def characterHello(name):  
  
    # Выводим приветствие с передачей аргумента  
    print("Привет, " + name + "!")
```

Далее:

```
# В переменной name (имя) мы вызываем функцию characterName
# и то, что возвращает функцию → переносится в эту переменную (name)
name = characterName()

# Вызываем функцию characterHello и передаем как параметр нашу прошлую переменную
# что хранит имя (name) нашего персонажа
characterHello(name)
```

Советую заикнуться на этом моменте и получше понять работу самого кода!

Вывод:

```
w1@MacBook-Air-W1 GuideToDef
Введите имя персонажа: Max
Привет, Max!
```

ДЗ - напишите подобную логику, но уже в рамках вашей фантазии!

Def в Python (№5)

“Функция и дружба с условиями”

*советуется изучать в связке с проектом на гитхабе
(<https://github.com/the-wlfrm/GuideToDef>)*

Что-то мы заикнулись на персонаже и запросами к нему!
Сейчас мы напишем функцию, что принимает наш возраст и проводит проверку на совершеннолетие

Представим, что мы хотим попасть в клуб
но на входе стоит охранник и просматривает паспорта
конечно-же, чтобы удостовериться в совершеннолети

И наша функцию будет выполнять ту же роль!

*Инициализируем/создаем нашу первую функцию def() и называем ее
"ageChecker"*

Он принимает у нас наш возраст, следовательно - параметр (age)

```
def ageChecker(age):  
  
    # Если возраст будет БОЛЬШЕ/РАВНО 18 : Вход разрешен! Вы совершеннолетний!  
    if age ≥ 18:  
        print("Вход разрешен! Вы совершеннолетний!")  
  
    # Иначе : Вы несовершеннолетний!  
    else:  
        print("Вы несовершеннолетний!")
```

Запрашиваем у пользователя возраст для дальнейшего использования

```
# Запрашиваем у пользователя указать возраст  
# Обратите внимание, я написал int(input())  
# это было сделано для более строгой проверки введенного пользователем  
# то есть, если пользователь введет слово, а не свой возраст, что в формате чисел  
# то программа выдаст ошибку! (используйте подобную конструкцию, если проверка строгая)  
age = int(input("Введите свой возраст: "))
```

Вызываем функцию и передаем возраст, что указал пользователь!

```
# Вызываем функцию и передаем туда то, что ввел пользователь  
ageChecker(age)
```

Вопрос для закрепления:

“ почему мы передаем параметр "age" при запуске функции? ”

Вывод программы такой:

```
w1@MacBook-Air-W1 GuideToDef % /usr/1
Введите свой возраст: 18
Вход разрешен! Вы совершеннолетний!
```

Помните, мы добавили выше такую конструкцию “int(input())” ?

Так что-же будет, если ввести буквы, а не цифры, как мы запрашивали?

```
w1@MacBook-Air-W1 GuideToDef % /usr/local/bin/python3 /Users/w1/GuideToDef/
Введите свой возраст: Max
Traceback (most recent call last):
  File "/Users/w1/GuideToDef/lessons/5_lesson.py", line 32, in <module>
    age = int(input("Введите свой возраст: "))
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
ValueError: invalid literal for int() with base 10: 'Max'
```

Python выдает ошибку о том, что мы запрашивали строго числовое значение, а не строковое и программа прекращает свою работу! (берите на заметку)

ДЗ - напишите свою функцию, что будет проверять возраст, но с более строгой проверкой!

Памятка для ДЗ:

== - СРАВНЕНИЕ

!= - ОТРИЦАТЕЛЬНОЕ СРАВНЕНИЕ

<= - МЕНЬШЕ/РАВНО

>= - БОЛЬШЕ/РАВНО

Ваша задача понять, где подобное можно использовать!

Def в Python (№6)

“Игра - вариация фантазии”

советуется изучать в связке с проектом на гитхабе
(<https://github.com/the-wlfrm/GuideToDef>)

Функции выполняют отдельные свои части кода, что могут нести разный смысл действий, но и иметь логическую цепочку действий и это играет нам на руку, ведь теперь мы можем написать мини-игру с помощью функций!

План такой:

- Мы создаем запросы к пользователю с сбором информации о нашем персонаже
- Создаем функцию для очистки терминала
- Создаем функцию для атаки босса
- Создаем функцию для лечения нашего персонажа

Также уточню, что мы используем 2 библиотеки для:

1. Установки времени перехода между функциями (import time)
2. Очистки терминала после выполнения действия (import os)

Давайте начнем!

Импортируем/Добавляем эти библиотеки/модули в наш файл

```
import os
import time
```

Инициализируем/создаем конструкцию для старта нашего проекта

```
# Инициализируем/создаем конструкцию для старта
if __name__ == "__main__":

    # Инициализировали/создали базовые значения
    boss_hp = 100
    character_hp = 100
```

(boss_hp = 100 (здоровье босса) ; character_hp = 100 (здоровье персонажа))

Создаем функцию, что просто при вызове будет отсчитывать 2 секунды и очищать терминал (Назовем "timeToClear")

```
def timeToClear():

    # time имеет метод .sleep(секунды), который временно задерживает код
    time.sleep(2)
    # os имеет метод .system(команда), которая может выполнить любую терминальную команду
    # ("cls") внутри метода - это буквально "clear" (то есть, все что в программе)
    # - мгновенно стирается
    os.system("cls")
```

Конструкция if __name__ == "__main__": - это конструкция, что проверяет, запущен ли файл напрямую как самостоятельная программа или импортирован в другой скрипт как модуль

Мы его добавили, что-бы просто удобно начать нашу программу!

Создаем функцию для атаки нашего босса!

```
# Создаем функцию для атаки босса
def attack():

    # global - это слово, которое указывает на то,
    # что переменную можно изменять даже тогда, когда она вне функции/не в функции
    global boss_hp

    # Запрашиваем пользователя об уроне, что он хочет нанести (строгая проверка записи)
    # - конструкция int(input())
    damage = int(input("Введите урон, что хотите нанести: "))

    # Вычитаем из здоровья босса наносимый нами урон
    boss_hp -= damage

    # Если здоровье босса МЕНЬШЕ/РАВНО 0 : Босс умер - мы победили!!!
    if boss_hp <= 0:
        print("Босс умер... Ты выиграл!!!")

        # Вызываем функцию для очистки
        timeToClear()

    # Иначе : атаквали босса - осталось здоровья
    else:

        # Прошу обратить внимание на " " + str(boss_hp) + " "
        # Мы использовали str(boss_hp), чтобы наше значение (число/int) перешло в (строку/str)
        # и мы смогли записать его в наш вывод, иначе была-бы ошибка!
        print("Ты атаковал босса (У босса " + str(boss_hp) + "hp)")

        # Вызываем функцию для очистки
        timeToClear()
```

Лучше хорошенько разобраться с кодом внутри гитхаба
(попросто лучше видно)

Заходите в браузер -> пишите -> <https://github.com/the-wlfrm/GuideToDef>

Создаем функцию выздоровления для нашего персонажа!

```
def heal():  
  
    # Все тоже самое, что и с функцией attack()  
    global character_hp  
  
    heal_amount = int(input("Введите на сколько hp хотите вылечить: "))  
    if character_hp ≥ 100:  
        print("У тебя и так 100hp, куда больше?")  
        timeToClear()  
    else:  
  
        if character_hp > 100:  
  
            # Мы указали, что если здоровье нашего персонажа будет больше 100 после лечения  
            # то сразу переопределяем его здоровье и превращаем в 100  
            # (просто легонько ввели ограничение без усложнений)  
            character_hp = 100  
  
        print("Ты вылечился на " + str(heal_amount) + "!")  
        timeToClear()
```

Ваша задача прямо сейчас, понять по первой функции, почему мы написали:

1. global character_hp
2. Проверку character_hp > 100:
3. heal_amount = int(input())
4. timeToClear()

Создаем начальное меню!

```
# Инициализируем/создаем конструкцию для старта
if __name__ == "__main__":

    # Инициализировали/создали базовые значения
    boss_hp = 100
    character_hp = 100

    # Конструкция " while True: "
    # - это просто бесконечный цикл, который вы в нашем случае
    # либо обрываете 3 опцией
    # либо ломаете ложными данными (на ваше понимание)
    while True:
        banner1 = "[1] Атаковать босса"
        banner2 = "[2] Вылечиться"
        banner3 = "[3] Выйти"
        print(banner1)
        print(banner2)
        print(banner3)

        choice = int(input("Выберите опцию: "))

        if choice == 1:
            attack()

        elif choice == 2:
            heal()

        elif choice == 3:
            print("Выходим...")
            time.sleep(2)

            # os._exit(0) - успешный выход из терминала
            os._exit(0)

        else:
            print("Такой опции не существует!")
```

Конструкция “ while True ”: нам нужно для бесконечного продолжения программы. Попробуйте прочитать этот код и понять его работу

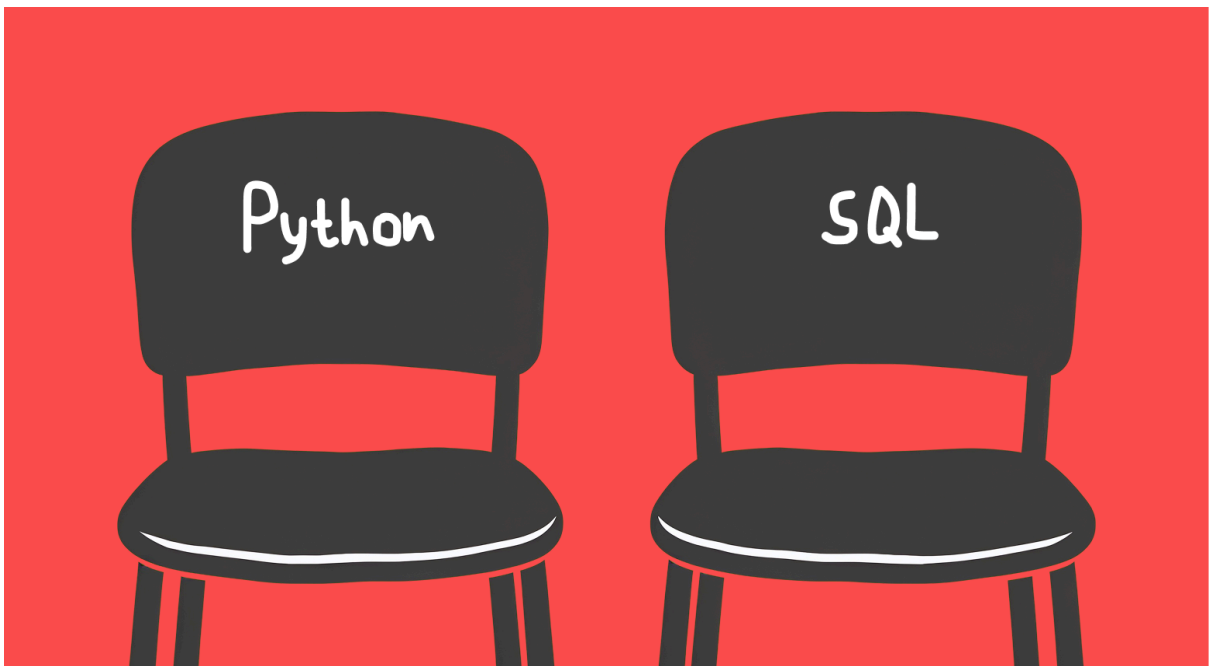
То есть, после каждого запуска функции, наш терминал очищается и снова создается меню до тех пор, пока вы сами не прекратите работу программы

ДЗ - Написать свою схватку, но передать и использовать в атаке и в здоровье такой параметр как - "Имя" (name)

КОНЕЦ!

Дальше самостоятельные задания
(не расслабляйся)

Красные стулья



Самостоятельная работа

def(), пацыки?

Начинающий уровень:

1. Приветствие

Создайте функцию `say_hello(name)`, которая принимает имя человека в виде строки и выводит на экран текст: Привет, <имя>!.

2. Сложение двух чисел

Напишите функцию `add_numbers(a, b)`, которая принимает два числа и возвращает их сумму (через `return`).

3. Проверка на четность

Напишите функцию `is_even(number)`, которая принимает целое число и возвращает `True`, если число четное, и `False`, если нечетное.

Средний уровень:

1. Поиск максимального числа

Создайте функцию `find_max(numbers)`, которая принимает список чисел и возвращает самое большое число из этого списка (не используя встроенную функцию `max()`).

2. Подсчет гласных букв

Напишите функцию `count_vowels(text)`, которая принимает строку и возвращает количество гласных букв (а, е, ё, и, о, у, ы, э, ю, я) в этой строке.

3. Проверка на палиндром

Создайте функцию `is_palindrome(word)`, которая принимает слово и возвращает `True`, если слово читается одинаково в обеих сторонах (например, «шалаш»), и `False` в противном случае.

* Далее продвинутый уровень! Листай. *

Продвинутый уровень:

1. Факториал числа

Напишите функцию `factorial(n)`, которая вычисляет факториал целого неотрицательного числа `n` с помощью цикла `for` или `while`.

2. Фильтрация списка

Создайте функцию `get_evens(numbers)`, которая принимает список чисел и возвращает новый список, состоящий только из четных элементов исходного списка.

Успехов!